

Curso de Go

Aula 6 - Arrays e Slices

Professor : Antônio Marcelo

NÚCLEA
ACADEMY

 `código_`
BRAZUCA ^[3]





Nessa Aula nos Vamos Ver

- Entender o conceito de Array
- Entender o conceito de Slice
- Entender diferenças, funcionamento interno e uso eficiente de memória
- Parte Prática



O que é um Array

- **Arrays têm tamanho fixo e fazem parte do tipo da variável.**
- **Características:**
 - **Tamanho fixo (não muda depois de criado)**
 - **O tamanho faz parte do tipo ([3]int é diferente de [4]int)**
 - **Quando você passa um array para uma função, ele é copiado**
 - **Mais rígido e pouco usado no dia a dia**

Exemplo de código

```
var a [3]int = [3]int{1, 2, 3}
```



O que é um Slice

- **Slices são estruturas dinâmicas que apontam para um array interno.**
- **Características:**
 - **Tamanho variável (pode crescer com append)**
 - **Não inclui tamanho no tipo ([]int)**
 - **Passado por referência (modifica o original)**
 - **Muito mais usado em Go**

Exemplo de Código:

```
s := []int{1, 2, 3}
```



Diferenças

- **Array**

- **tamanho fixo**
- **cópia ao passar**
- **pouco usado**

Exemplo de código

```
func muda(a [3]int) {  
    a[0] = 100  
}
```

```
func main() {  
    a := [3]int{1, 2, 3}  
    muda(a)  
    fmt.Println(a) // continua [1 2 3]  
}
```

- **Slice**

- **tamanho dinâmico**
- **referência para array**
- **padrão em Go**

Exemplo de código

```
func muda(s []int) {  
    s[0] = 100  
}
```

```
func main() {  
    s := []int{1, 2, 3}  
    muda(s)  
    fmt.Println(s) // vira [100 2 3]  
}
```




Comprimento vs capacidade

- **Em slices, existem dois conceitos fundamentais:**
 - **len** → quantos elementos o slice possui agora
 - **cap** → quantos elementos ele pode armazenar antes de precisar crescer

Exemplo de código

```
nums := []int{1, 2, 3}
fmt.Println(len(nums)) // 3
fmt.Println(cap(nums)) // 3
```

- **Explicação:**
 - **O slice tem 3 elementos → len = 3**
 - **A capacidade inicial também é 3 → cap = 3**
 - **Se você adicionar mais elementos além da capacidade, Go precisará realocar memória**



Comprimento vs capacidade

- **Outro exemplo**

Exemplo de código

```
nums := make([]int, 2, 5)
fmt.Println(len(nums)) // 2
fmt.Println(cap(nums)) // 5
```

- **Explicação:**

- **O slice começa com 2 elementos**
- **Mas já tem espaço reservado para até 5**
→ evita realocações imediatas



Crescimento com append

- **A função append adiciona elementos ao slice e retorna um novo slice**

Exemplo de código

```
nums := []int{1, 2, 3}
nums = append(nums, 4, 5)
fmt.Println(nums) // [1 2 3 4
5]
```

- **O que acontece internamente:**
 - **Se ainda houver capacidade disponível → o mesmo é reutilizado**
 - **Se não houver espaço → Go cria um novo maior e copia os dados**

- **Boa prática:**
 - **Sempre reatribuir o resultado de append**
 - **Use make com capacidade quando souber o tamanho esperado**



Compartilhamento de memória

- O compartilhamento de memória em Go aparece principalmente quando você trabalha com slices, mapas e ponteiros. No caso de slices, isso é especialmente importante porque pode gerar efeitos inesperados se você não entender bem o comportamento.
- Um slice não guarda os dados diretamente. Ele é uma estrutura leve que aponta para:
 - um array interno (onde estão os dados)
 - o comprimento (len)
 - a capacidade (cap)
- Quando você cria novos slices a partir de outro, eles continuam apontando para o mesmo array base.



Slices e Memória

- **Como funciona nos slices**

Exemplo de código

```
base := []int{1, 2, 3, 4}
```

```
a := base[0:2] // [1 2]
```

```
b := base[1:3] // [2 3]
```

- **Aqui, a e b não são cópias. Ambos compartilham o mesmo conteúdo interno de base.**



Quando o append muda tudo

- O comportamento muda quando ocorre realocação:

Exemplo de código

```
s := make([]int, 0, 2)

s = append(s, 1, 2)
t := s

s = append(s, 3) // pode criar novo
array

s[0] = 100

fmt.Println(s) // [100 2 3]
fmt.Println(t) // [1 2]
```

- Aqui:
 - **t continua apontando para o array antigo**
 - **s pode passar a apontar para um novo array**
 - **A partir desse momento, eles deixam de compartilhar memória**
 - **Esse é um ponto crítico em Go.**



Pontos importantes

- **Problemas comuns:**
 - **Alterações inesperadas em dados compartilhados**
 - **Funções modificando slices “sem querer”**
 - **Bugs difíceis de rastrear**
- **Boas práticas**
 - **Assuma que slices compartilham memória por padrão**
 - **Documente funções que modificam slices**
 - **Use copy quando precisar de segurança**
 - **Tenha atenção com append e capacidade**
- **Slices são eficientes justamente por compartilharem memória, mas isso exige disciplina.**
- **Entender quando você está compartilhando e quando está copiando é essencial para evitar bugs e escrever código previsível.**



Pontos importantes

- **Problemas comuns:**
 - **Alterações inesperadas em dados compartilhados**
 - **Funções modificando slices “sem querer”**
 - **Bugs difíceis de rastrear**
- **Boas práticas**
 - **Assuma que slices compartilham memória por padrão**
 - **Documente funções que modificam slices**
 - **Use copy quando precisar de segurança**
 - **Tenha atenção com append e capacidade**
- **Slices são eficientes justamente por compartilharem memória, mas isso exige disciplina.**
- **Entender quando você está compartilhando e quando está copiando é essencial para evitar bugs e escrever código previsível.**



Comando Make

- O make em Go é uma ferramenta essencial para trabalhar com estruturas dinâmicas. Ele é usado para inicializar slices, maps e channels, preparando memória interna para uso imediato.
- O ponto mais importante:
- make não cria “tipos”, ele inicializa estruturas que já possuem comportamento interno (diferente do new).
- O uso mais comum é com slices, permitindo controlar tamanho e capacidade.



Criando slices com make

- O **make** permite definir capacidade

Exemplo de código

```
nums := make([]int, 2, 5)
fmt.Println(len(nums)) // 2
fmt.Println(cap(nums)) // 5
```

- **Útil para otimização**



Uso eficiente de memória

- **Boas práticas:**
 - **Evite crescimento excessivo com append**
 - **Use make quando souber o tamanho esperado**
 - **Evite re-alocações desnecessárias**
 - **Cuidado ao compartilhar slices**

Exemplo de código

```
nums := make([]int, 0, 100)

for i := 0; i < 100; i++ {
    nums = append(nums, i)
}
```

The screenshot shows a Go IDE with a file named 'Untitled-1' in the 'package main' directory. The code defines a struct 'rect' with 'width' and 'height' of type 'int'. It includes two methods: 'area()' which returns the area, and 'perim()' which returns the perimeter. The 'main' function creates a 'rect' with width 10 and height 5, prints its area and perimeter, and then demonstrates passing a pointer to the 'rect' to the same methods. Comments in Portuguese explain the methods and the pointer usage. The bottom status bar indicates the terminal is running as administrator.

```
1 package main
2 import "fmt"
3 type rect struct {
4     width, height int
5 }
6 Este método area possui um tipo receptor de *rect.
7
8 func (r *rect) area() int {
9     return r.width * r.height
10 }
11 Métodos podem ser definidos para qualquer tipo de receptor ponteiro ou valor. Aqui temos um exemplo de receptor
12
13 func (r rect) perim() int {
14     return 2*r.width + 2*r.height
15 }
16 func main() {
17     r := rect{width: 10, height: 5}
18     Nós podemos chamar 2 métodos definidos na nossa estrutura.
19
20     fmt.Println("area: ", r.area())
21     fmt.Println("perim:", r.perim())
22     Go trata automaticamente conversões entre valores e ponteiros para métodos de chamada. Você pode querer usar um
23
24     rp := &r
25     fmt.Println("area: ", rp.area())
26     fmt.Println("perim:", rp.perim())
27 }
```



Parte Prática

Praticar Arrays e Slices